

Universidad del Valle de Guatemala

Construcción de Compiladores – CC-3032, sección 10

Compiscript Semantic IDE

Proyecto 1: análisis semántico, tabla de símbolos e IDE



Grupo DragonsSlayers 2.0

Pablo Daniel Barillas Moreno 22193

Hugo Daniel Barillas Ajín 23556

Ernesto Ascencio 23009

Catedrático: Ing. Carlos Valdéz

Agosto de 2026

Índice

1	Resumen ejecutivo	2
2	Objetivos	2
2.1	Objetivo general	2
2.2	Objetivos específicos	2
3	Contexto y fuentes del lenguaje	2
4	Arquitectura	3
4.1	Pipeline por fases	3
4.2	Responsabilidades de módulos	3
5	Sistema de tipos	3
5.1	Reglas principales	5
5.2	Inferencia de retornos	5
6	Tabla de símbolos y ámbitos	5
6.1	Operaciones requeridas por la rúbrica	5
6.2	Identidad léxica de clases	6
7	Funciones, closures y clases	6
8	Control de flujo	6
9	Diagnósticos	7
10	Diseño del IDE	7
11	Auditoría y correcciones	8
12	Pruebas y resultados	8
13	Correspondencia con la rúbrica	9
14	Limitaciones y trabajo futuro	9
15	Conclusiones	9
A	Programa breve de demostración	10
B	Documentos de la entrega	10

1 Resumen ejecutivo

Este proyecto implementa un entorno integrado para Compiscript, un subconjunto inspirado en TypeScript. La aplicación recibe código fuente, ejecuta el lexer y parser generados con ANTLR 4 y, cuando la sintaxis es válida, recorre el CST mediante un Visitor para aplicar reglas semánticas. El resultado incluye diagnósticos con ubicación, árbol semántico anotado, tabla de símbolos, árbol de ámbitos, referencias y capturas de closures.

La solución fue construida con TypeScript, React, Vite y Electron. El diseño separa el motor de compilación de la presentación: la interfaz, la CLI y las pruebas invocan el mismo orquestador. La versión final corrige problemas de identidad y alcance de clases, inferencia dependiente del orden, firmas sin retorno explícito, conteo incompleto de referencias y ejemplos incompatibles con la gramática formal.

La verificación final reporta **105 pruebas aprobadas en 6 suites**, además de compilación TypeScript y build de producción exitosos. La matriz completa se encuentra en `docs/MATRIZ_REQUISITOS.md`.

2 Objetivos

2.1 Objetivo general

Construir una fase de análisis semántico para Compiscript que sea comprobable mediante pruebas, observable desde un IDE y suficientemente modular para servir como base de fases posteriores del compilador.

2.2 Objetivos específicos

1. Generar el lexer y parser a partir de una gramática ANTLR.
2. Implementar reglas de tipos, ámbitos, funciones, flujo, clases y arreglos.
3. Construir una tabla de símbolos con inserción, recuperación, actualización y manejo de alcances.
4. Producir un árbol semántico visual con tipos y referencias.
5. Presentar resultados comprensibles en un IDE web y de escritorio.
6. Mantener pruebas de éxito, fallo y regresión para las reglas requeridas.
7. Documentar decisiones donde las fuentes del lenguaje son ambiguas.

3 Contexto y fuentes del lenguaje

El desarrollo parte de tres fuentes: el README general de Compiscript, la gramática `Compiscript.g4` y el enunciado de análisis semántico. La gramática es la autoridad formal para la sintaxis; el enunciado define las validaciones; y los ejemplos ayudan a interpretar la intención del lenguaje.

Se identificaron dos discrepancias importantes:

- El enunciado requiere `float`, pero la gramática base solo declara `integer`. Se añadió el token de tipo y el literal decimal como extensión mínima.
- El enunciado agrupa `switch` con condiciones booleanas, mientras el README muestra casos enteros y la gramática acepta una expresión general. Se adoptó un discriminante escalar y se exige comparabilidad entre discriminante y casos.

La gramática formal exige bloques con llaves para `if`, `while`, `do-while`, `for` y `foreach`. La versión activa mantiene esa restricción aun cuando algunos fragmentos narrativos del README omiten llaves.

4 Arquitectura

4.1 Pipeline por fases



Figura 1: Representación conceptual del pipeline de Compiscript.

1. **Lexer:** transforma caracteres en tokens y registra errores sin detener innecesariamente la exploración.
2. **Parser:** ejecuta `program()` y produce el CST con recuperación controlada de errores.
3. **Prepasada:** crea identidades de clase y prepara declaraciones visibles en cada ámbito.
4. **Visitor semántico:** resuelve nombres, calcula tipos, valida reglas y construye el árbol anotado.
5. **Presentación:** organiza un único `AnalyzeResult` para UI, CLI y exportaciones.

La semántica queda en estado `skipped` si existen errores léxicos o sintácticos. Esto evita cascadas producidas por contextos recuperados artificialmente por el parser.

4.2 Responsabilidades de módulos

5 Sistema de tipos

El modelo representa tipos primitivos, arreglos, funciones, clases, instancias, `null`, `void`, `unknown` y `error`. Los dos últimos permiten continuar el análisis sin multiplicar mensajes derivados.

Módulo	Responsabilidad
<code>lib/analyze.ts</code>	Orquestación de lexer, parser, semántica y resultado común.
<code>semantic/declarationVisitor.ts</code>	Registro de clases, miembros, firmas y herencia.
<code>semantic/semanticVisitor.ts</code>	Recorrido principal del CST y construcción del árbol semántico.
<code>semantic/scopes.ts</code>	Tabla de símbolos, cadena léxica y actualización controlada.
<code>semantic/typeSystem.ts</code>	Asignabilidad, comparación, operadores e inferencia común.
<code>semantic/flowAnalysis.ts</code>	Terminación, retorno por caminos y código inalcanzable.
<code>semantic/diagnostics.ts</code>	Códigos, severidad, ubicación y deduplicación.
<code>ui/</code>	Representación interactiva de resultados; no contiene reglas del lenguaje.

Cuadro 1: Separación de responsabilidades.

Familia	Caso válido	Rechazo característico
Aritmética	operandos numéricos; <code>string + string</code>	booleanos, arreglos o funciones
Lógica	operandos <code>boolean</code>	cualquier valor no booleano
Relacional	números compatibles	categorías incompatibles
Asignación	igualdad o promoción <code>integer</code> a <code>float</code>	conversión implícita inversa
Arreglos	elementos con tipo común	mezcla sin tipo compatible
Índice	receptor arreglo e índice entero	receptor no indexable o índice no entero

5.1 Reglas principales

```
let contador = 1;  
let promedio: float = contador;  
let valores: integer[] = [1, 2, 3];  
print(valores[0]);
```

5.2 Inferencia de retornos

Una función sin anotación empieza con tipo `unknown`. Cada `return` registra el tipo observado y al terminar el cuerpo se calcula un tipo común. Una función sin retornos con valor resulta `void`. Este tratamiento evita el error previo de considerar toda función no anotada como procedimiento.

6 Tabla de símbolos y ámbitos

Cada símbolo almacena ID, nombre, categoría, tipo, mutabilidad, ámbito, posición de declaración, inicialización, referencias, captura y, cuando corresponde, firma o miembros.

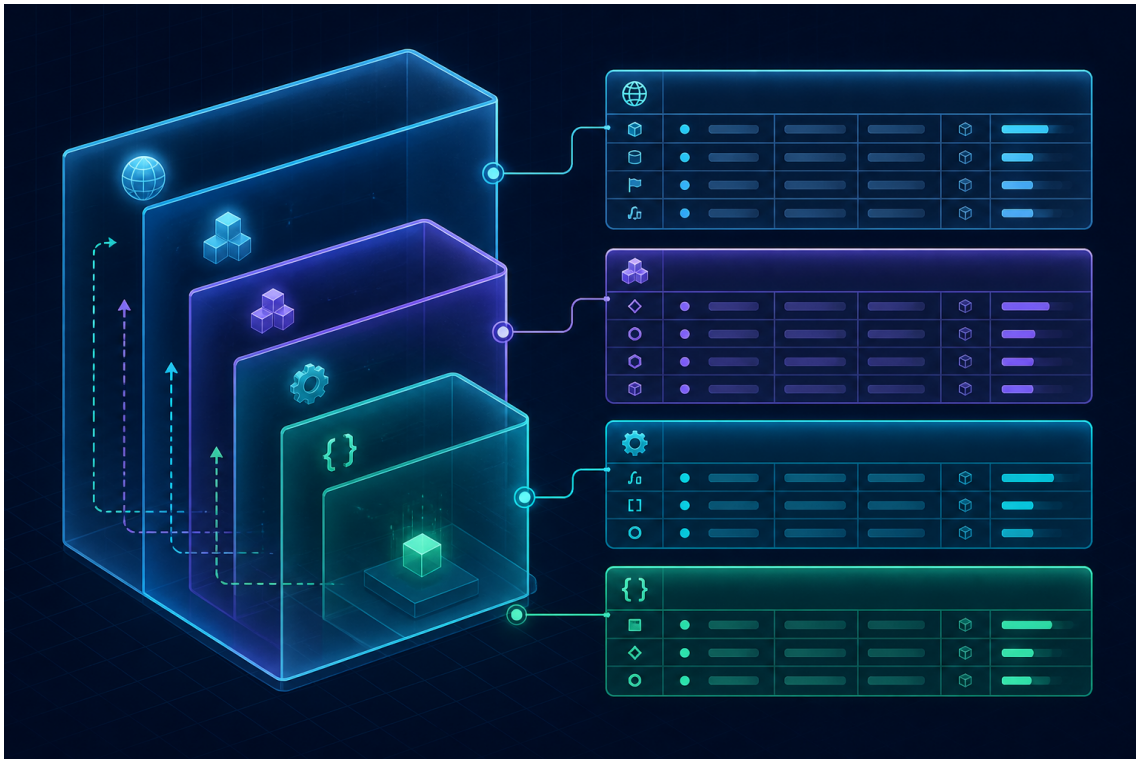


Figura 2: Modelo conceptual de ámbitos anidados y tabla de símbolos.

6.1 Operaciones requeridas por la rúbrica

El shadowing se permite en ámbitos hijos. La actualización controlada evita reemplazar accidentalmente el ID, el nombre o el ámbito de un símbolo. Inicialización y captura reutilizan la misma operación.

Operación	API	Garantía
Insertar	<code>declare</code>	rechaza duplicados en el ámbito actual
Recuperar	<code>resolveCurrent</code> , <code>resolve</code>	búsqueda local o recorrido de padres
Actualizar	<code>updateSymbol</code>	preserva identidad y ubicación de declaración
Alcances	<code>enterScope</code> , <code>exitScope</code>	conserva el árbol y restaura el padre

6.2 Identidad léxica de clases

La versión revisada eliminó el registro global por nombre. Cada declaración de clase recibe un `classId`; el nombre visible se declara en el `ScopeManager`. Los tipos de clase e instancia conservan ese ID. Por tanto, dos clases `Local` declaradas en bloques hermanos son distintas y ninguna es visible fuera de su bloque.

7 Funciones, closures y clases

Las funciones y clases se registran antes de recorrer las demás instrucciones del ámbito. Este hoisting local permite recursión y referencias adelantadas sin convertir las declaraciones locales en globales.

```
function factorial(n: integer): integer {
  if (n <= 1) { return 1; }
  return n * factorial(n - 1);
}
```

```
function crear(): integer {
  let base: integer = 40;
  function sumar(): integer {
    return base + 2;
  }
  return sumar();
}
```

Una referencia desde una función anidada a un símbolo de una función externa marca el símbolo como capturado. La UI lo presenta en la tabla y en las métricas.

En clases se validan constructor explícito o implícito, herencia, ciclos, `this`, miembros y constantes. Los campos se recorren antes de los métodos para que la inferencia no dependa del orden textual. El árbol final recupera el orden del programa para conservar legibilidad.

8 Control de flujo

Las condiciones de `if` y ciclos deben ser booleanas. `return` solo es válido dentro de una función; `continue` solo dentro de ciclos; y `break` dentro de ciclos o `switch`. Las instrucciones posteriores a una terminación se reportan con SEM018 como advertencia.

`flowAnalysis.ts` separa dos preguntas: si una instrucción termina el flujo inmediato y si una función retorna en todos los caminos. Esta separación reduce falsos positivos en bloques e instrucciones condicionales.

9 Diagnósticos

El catálogo comprende SEM001 a SEM023; SEM022 permanece reservado para compatibilidad y SEM023 distingue una variable existente pero aún no inicializada. Cada mensaje contiene fase, severidad, línea, columna, descripción y sugerencia cuando aplica.

Código	Regla
SEM001	Identificador no declarado.
SEM002	Redeclaración en el mismo ámbito.
SEM003	Asignación o inicialización incompatible.
SEM004	Operador aplicado a tipos inválidos.
SEM005	Condición no booleana.
SEM006	Cantidad incorrecta de argumentos.
SEM007	Tipo de argumento incompatible.
SEM008	Tipo de retorno incompatible.
SEM009	Retorno fuera de una función.
SEM010	<code>break</code> fuera de ciclo o <code>switch</code> .
SEM011	<code>continue</code> fuera de ciclo.
SEM012	Miembro inexistente.
SEM013	Uso inválido de <code>this</code> .
SEM014	Clase, constructor o invocación inválida.
SEM015	Índice no entero.
SEM016	Valor no indexable.
SEM017	Arreglo heterogéneo.
SEM018	Código inalcanzable.
SEM019	Parámetro duplicado.
SEM020	Herencia inválida o circular.
SEM021	Discriminante o caso incompatible en <code>switch</code> .
SEM022	Reservado para compatibilidad.
SEM023	Uso antes de inicialización.

10 Diseño del IDE

La refactorización visual busca que el usuario comprenda la fase antes de interpretar tablas extensas. El encabezado resume el pipeline y el estado. En semántica, una guía lateral explica el propósito de cada etapa; el editor queda como superficie principal; y los resultados se distribuyen en pestañas.

- **Resumen:** métricas de salud, símbolos, referencias, ámbitos y closures.
- **Diagnósticos:** filtros y ubicación de cada error o advertencia.
- **Símbolos:** búsqueda, categorías, tipos, mutabilidad y exportación.
- **Ámbitos:** jerarquía navegable de entornos.

- **Árboles:** CST y árbol semántico comparados en una cuadrícula.

El editor incorpora carga de archivos, copia, posición del cursor, estado de preparación y atajos. Se eliminaron emojis del código y documentación; la iconografía funcional usa componentes vectoriales consistentes.

11 Auditoría y correcciones

Área	Hallazgo	Corrección
Clases	Registro global por nombre filtraba clases locales y chocaba con homónimos.	Identidad <code>classId</code> y resolución por tabla léxica.
Campos	Inferencia dependía del orden entre campos y métodos.	Campos primero; presentación restaurada al orden fuente.
Funciones	Falta de anotación se interpretaba como <code>void</code> .	Recolección e inferencia de retornos observados.
Referencias	Accesos a clase, campo y método no incrementaban todos los símbolos.	Propagación de <code>symbolId</code> y registro en cada acceso.
Tabla	No existía una actualización pública y verificable.	<code>updateSymbol</code> con invariantes y pruebas directas.
Gramática	Cuerpos de control aceptaban instrucciones sin bloque.	Reglas alineadas con <code>Compiscript.g4</code> oficial y regeneración ANTLR.
Ejemplos	Casos integrados conservaban <code>if</code> sin llaves.	Ejemplos y pruebas sincronizados con la gramática.
UI	Resultados largos competían visualmente con el editor.	Workbench, guía lateral y explorador por pestañas.
Documentación	Comandos desde una raíz incorrecta y cifras antiguas.	Rutas, matriz, informe y presentación actualizados.

12 Pruebas y resultados

La batería incluye pruebas unitarias del `ScopeManager`, reglas semánticas, pipeline integral, ejemplos sincronizados y archivos preparados para la rúbrica.

Los comandos finales son:

```
cd compiscript-ide-work
npm install
npm run generate
npm run check
npm test
npm run build
```

Suite	Pruebas aprobadas
<code>semanticAnalyzer.test.ts</code>	68
<code>scopeManager.test.ts</code>	4
<code>analyze.compiscript.test.ts</code>	14
<code>rubric.examples.test.ts</code>	8
<code>examples.sync.test.ts</code>	3
<code>projectExamples.test.ts</code>	8
Total	105

En la revisión final, `npm run check`, `npm test` y `npm run build` terminaron correctamente. Vite informa una advertencia no bloqueante sobre el tamaño del bundle principal; no afecta la corrección, aunque puede mitigarse posteriormente con división dinámica de módulos.

13 Correspondencia con la rúbrica

Componente	Puntos	Evidencia
IDE	15	Editor, navegación por fases, guía, resultados interactivos y exportaciones.
Parser y semántica	60	ANTLR, Visitor, sistema de tipos, 23 códigos, árbol y pruebas.
Tabla de símbolos	25	Inserción, recuperación, actualización, alcances, referencias y closures.

La contribución individual no puede inferirse desde el código final: debe demostrarse con commits reales del repositorio. No se alteró ni documentó autoría que el historial no pueda respaldar.

14 Limitaciones y trabajo futuro

- La política de `switch` debe confirmarse con el docente debido a la contradicción entre fuentes.
- El bundle web puede dividirse para reducir el tamaño de descarga inicial.
- El análisis de retorno puede evolucionar hacia un grafo de control de flujo completo.
- La tabla de símbolos está preparada para alimentar una representación intermedia, pero esa fase no pertenece a esta entrega.
- Las referencias permiten métricas y navegación futura hacia declaración.

15 Conclusiones

El resultado no se limita a detectar errores: expone cómo el compilador interpreta el programa. La tabla de símbolos permite observar identidad y alcance; el árbol semántico relaciona sintaxis y tipos; los diagnósticos ofrecen evidencia localizada; y las pruebas convierten las reglas del enunciado en comportamiento reproducible.

La corrección de clases fue el cambio arquitectónico central. Separar nombre visible e identidad de declaración elimina fugas entre ámbitos y permite razonar correctamente sobre herencia e instancias. La inferencia independiente del orden y el conteo de referencias completan una representación más coherente para futuras fases.

Finalmente, la nueva interfaz presenta el pipeline de forma progresiva y evita mostrar simultáneamente todas las tablas. Junto con el informe, la matriz y la presentación HTML, la entrega queda preparada tanto para evaluación técnica como para exposición oral.

A Programa breve de demostración

```
class Caja {
  let valor = 40;

  function sumar(delta: integer) {
    return this.valor + delta;
  }
}

function ejecutar(): integer {
  let caja: Caja = new Caja();
  return caja.sumar(2);
}

print(ejecutar());
```

Este programa demuestra clase, campo inferido, método sin retorno anotado, constructor implícito, invocación tipada y referencias a clase, campo y método.

B Documentos de la entrega

- README.md: ejecución, capacidades y rutas.
- docs/ARQUITECTURA_PROYECTO_1.md: diseño interno.
- docs/DECISIONES_SEMANTICAS.md: políticas e inconsistencias resueltas.
- docs/AUDITORIA_PROYECTO_1.md: hallazgos y validación.
- docs/MATRIZ_REQUISITOS.md: trazabilidad del enunciado.
- presentation/compiscript-proyecto-1.html: exposición interactiva.